

Enterprise RAG System: Complete Technical Walkthrough

1. PROJECT OVERVIEW

What this RAG project does

This project is an Enterprise-grade **Retrieval-Augmented Generation (RAG)** AI system. It allows users to ask questions about internal company documents (PDFs, text files, etc.) and get highly accurate, context-aware answers. If the AI cannot find the answer in the internal documents, it dynamically falls back to an Agentic Web Search to find the answer on the live internet.

Why RAG architecture is used & The Problem Solved

Large Language Models (LLMs) are trained on public data up to a certain date. They do not know your private company secrets, financial records, or real-time news. If you ask a standard LLM about an internal company policy, it will either say "I don't know" or worse, **hallucinate** (make up a fake answer). RAG solves this by physically fetching your internal documents and handing them to the LLM *before* it answers, grounding the AI in factual reality.

High-Level Architecture

1. **Ingestion Pipeline:** Reads PDFs, chops them into chunks, converts text to vectors, and stores them securely in a Vector Database.
 2. **Retrieval Pipeline:** Takes a user question, searches the Vector Database for relevant chunks, and feeds those chunks to the LLM.
 3. **Agentic Web Fallback:** If the database lacks the answer, the script queries DuckDuckGo for live internet data and feeds that to the LLM instead.
 4. **LLM Generation:** The vLLM server processes the prompt and returns the final answer.
-

2. COMPLETE TECHNOLOGY STACK EXPLANATION

AI / LLM Stack

- **LangChain:** Used as the foundational framework to wire different AI components together. We use LangChain's `HuggingFaceEmbeddings` to convert text to vectors and `SemanticChunker` to intelligently split documents.
- **LangGraph:** Used in the ingestion pipeline (`ingest.py`). Unlike LangChain (which is a straight line), LangGraph allows us to build complex, stateful, multi-step workflows (Nodes and Edges). We use it to strictly control the order of operations: Load Document -> Load Embedding Model -> Chunk Documents -> Store Vectors.
- **LangFuse:** An open-source observability platform. We use the Langfuse OpenAI wrapper to automatically capture **Tracing**. It tracks the exact prompts sent, the LLM's response, token usage, cost, and latency (how many milliseconds it took), giving admins full visibility into employee AI usage.
- **Hugging Face:** Used to source the open-source embedding models. We download and run the model entirely locally to ensure data privacy.

The LLM

- **Model:** We utilize lightweight instruct models (like `Qwen2.5-1.5B` or `Llama-3.2-1B-Instruct`). These were selected because they are incredibly fast and can run locally on standard consumer GPUs without sending data to the cloud.
- **vLLM Engine:** We do not run the model via standard Python. We run it inside a highly optimized **vLLM Docker Container**. vLLM uses "PagedAttention" (which manages the KV Cache like an operating system manages virtual memory). This allows continuous batching of requests, resulting in 10x-20x faster generation speeds than standard runners like Ollama.

Embedding System

- **Model Used:** `all-MiniLM-L6-v2`
- **Why it is required:** Computers don't understand words; they only understand numbers. An embedding model translates sentences into mathematical coordinates (vectors) in a 384-dimensional space. Sentences with similar meanings are placed physically closer together in this mathematical space, allowing the database to perform "Similarity Search" to find relevant paragraphs.

3. DOCUMENT INGESTION PIPELINE

(`ingestion/ingest.py`)

Workflow: Document Upload → Document Reading (Docling) → Semantic Chunking → Embedding Generation → Vector Storage

- **What is Ingestion?** The one-time process of reading raw files and formatting them so the AI can search them.
- **Document Loader (IBM Docling):** We discarded basic text rippers and implemented IBM Docling. It uses AI vision models (RapidOCR) to understand the physical layout of PDFs, perfectly parsing tables, charts, and 2-column layouts.
- **Chunking Strategy (Semantic Chunking):** Instead of blindly chopping documents every 500 characters, we use `SemanticChunker` with a `breakpoint_threshold_type="percentile"`. It uses the embedding model to analyze the meaning of sentences and only splits the document when the *topic* changes. This ensures chunks contain complete, unbroken thoughts.

4. VECTOR DATABASE (MILVUS)

- **What it is:** We use **Milvus**, a highly scalable, purpose-built Vector Database.
- **Why not SQL?** SQL databases look for exact keyword matches. Vector databases perform mathematical nearest-neighbor searches, allowing you to ask "What are the company's financial goals?" and retrieve a document that says "Our fiscal target is \$1M," even though the words don't match.
- **Schema:** We defined a strict schema (`rag_knowledge` collection) storing the 384-dimension vector alongside crucial metadata: `text`, `source`, `file_path`, and `required_role`.
- **Search Algorithm:** We utilize **AUTOINDEX** with **COSINE Similarity**. Cosine similarity measures the angle between two vectors. If the angle is small (cosine approaches 1), the text is highly relevant to the question.

5. RETRIEVAL PIPELINE (`retrieval/retrieve.py`)

Workflow: User Question → Question Embedding → Milvus Search → Relevant Chunk Retrieval → Prompt Creation → LLM Response

- **What is Retrieval?** The real-time process of finding the right data to answer a specific user question.
- **Top-K Search:** When the user asks a question, we convert it to a vector and ask Milvus to return the K closest vectors. We configured `k=5`, meaning we retrieve the 5 most semantically relevant chunks to build the LLM's context.

6. PROMPTING AND GENERATION FLOW

To prevent hallucination, we utilize a strict **System Prompt** architecture via the vLLM API:

Final Prompt Structure:

```
[
  {
    "role": "system",
    "content": "You are an AI assistant. Answer ONLY using the information",
  },
  {
    "role": "user",
    "content": "Information:\n[INSERTED MILVUS CHUNKS HERE]\n\nQuestion:\n"
  }
]
```

By isolating the rules in the "System" role, the LLM is forced to obey the anti-hallucination constraints before reading the user's question.

7. SECURITY IMPLEMENTATION (RBAC)

- **What is RBAC?** Role-Based Access Control ensures employees only see documents they are authorized to see.
 - **Implementation:**
 - During ingestion, `ingest.py` tags chunks. If the filename contains the word "secret", the chunk metadata `required_role` is set to `"admin"`. Otherwise, it is `"user"`.
 - During retrieval, `retrieve.py` checks `CURRENT_USER_ROLE`.
 - We inject a hard filter directly into the Milvus query: `filter=f"required_role in {allowed_roles}"`.
 - **Result:** If a `"user"` asks for a secret password, the database physically blocks the retrieval. The LLM never even sees the secret document.
-

8. BACKEND ARCHITECTURE

- **Current Implementation:** This project is intentionally built as two highly optimized, standalone Python CLI scripts (`ingest.py` and `retrieve.py`). There is currently no REST API framework (like FastAPI or Flask) wrapping the Python logic.
 - **Why?** This allows for rapid prototyping, direct terminal testing, and avoiding network overhead during local development.
 - **Environment:** All configuration is strictly managed via a `.env` file loaded using `python-dotenv`.
-

9. FRONTEND / UI (Open WebUI)

- **What it is:** Open WebUI is a ChatGPT-clone interface running locally.
 - **Architecture Note:** Open WebUI connects directly to the raw vLLM server via the OpenAI-compatible API. It is used to give employees a beautiful interface for general AI chat.
 - **Separation of Concerns:** Open WebUI *bypasses* our custom Python scripts. To use PDFs in Open WebUI, users must use its built-in native document uploader. Our custom Milvus/Docling/RBAC logic is restricted to our terminal architecture.
-

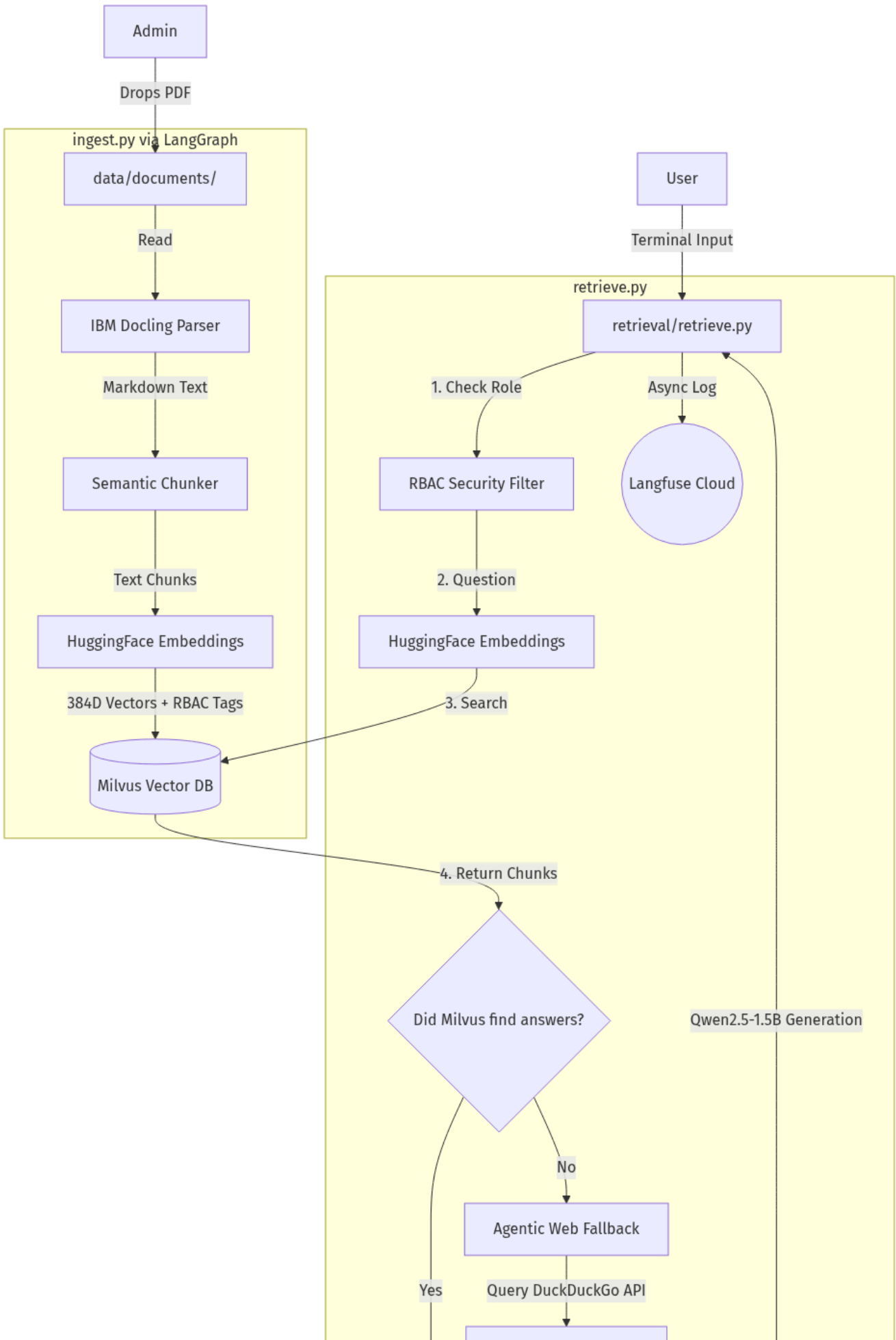
10. DOCKER AND DEPLOYMENT

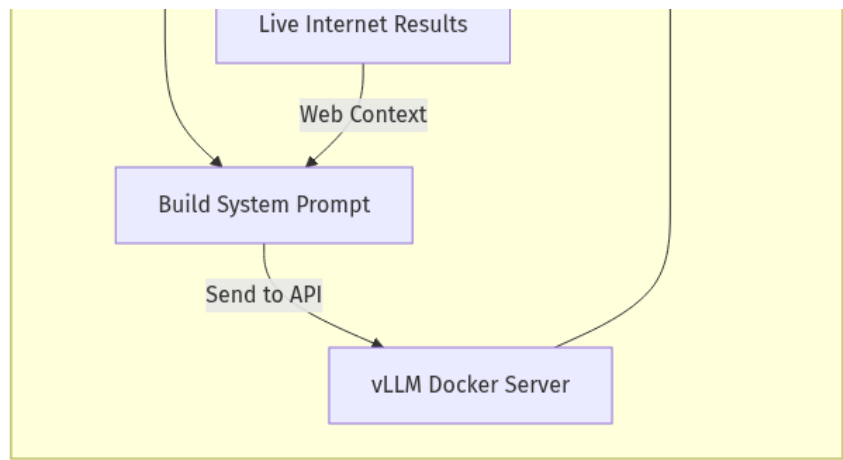
- **Why Docker?** Docker eliminates "it works on my machine" issues by packaging software into isolated containers.
 - **docker-compose.yml:** We use Docker Compose to orchestrate our infrastructure. Running `docker compose up -d` simultaneously spins up:
 - **Milvus Standalone Container:** The vector database.
 - **etcd & MinIO Containers:** Required dependencies for Milvus storage.
 - **vLLM Container:** The GPU-accelerated AI inference engine exposed on port 8000.
-

11. ENVIRONMENT AND DEPENDENCY FILES

- **.env:** Keeps hardcoded secrets out of the codebase. It stores `VLLM_API_BASE`, `MILVUS_URI`, and the `LANGFUSE_SECRET_KEY`.
 - **requirements.txt:** A highly curated, minimal dependency file containing only: `docling`, `sentence-transformers`, `langfuse`, and `duckduckgo-search`.
 - **.venv:** An isolated Python Virtual Environment. By installing dependencies here instead of globally on the OS, we permanently solved "Dependency Hell" and C++ compiler conflicts.
-

12. COMPLETE END-TO-END WORKFLOW DIAGRAM





13. PROJECT EXECUTION FLOW

When a user asks a question: 1. The script reads `CURRENT_USER_ROLE`. 2. The question is converted to a vector array. 3. Pymilvus performs a similarity search, returning the top 5 chunks *only* if the chunk's `required_role` matches the user's permissions. 4. If the LLM generates an answer containing "I don't have that information", the script triggers the fallback. 5. The script queries `duckduckgo-search` (DDGS) to pull 10 snippets from the live internet. 6. A strict anti-hallucination prompt is built using the web context and sent to vLLM. 7. The answer is printed to the terminal. 8. The Langfuse SDK silently transmits the trace (prompt, latency, tokens) to the cloud dashboard.

14. IMPROVEMENTS AND FUTURE UPGRADES

1. **FastAPI Backend:** Wrap `retrieve.py` in a FastAPI server to expose endpoints, allowing us to connect custom web frontends directly to our enterprise logic.
2. **Hybrid Search:** Upgrade Milvus to use both Vector Search (Cosine) and Keyword Search (BM25) simultaneously to improve retrieval accuracy for highly specific acronyms or part numbers.
3. **Reranking:** Implement a Cross-Encoder model (like `bge-reranker`) after Milvus retrieval to re-score and sort the top 5 chunks before feeding them to the LLM.

4. **Larger LLM:** Upgrade from the 1.5B parameter model to a 7B or 8B model (like Llama-3-8B-Instruct) to significantly enhance logical reasoning and reduce hallucination during Agentic Web Searches.